

LangGraph Workflow Design Document (LGWD)

IntelliPlant AI

AI-Powered Industrial Knowledge Intelligence Platform

Version: 1.0

Document Type: LangGraph Workflow Design

Prepared For: ET AI Hackathon 2026

Prepared By: Team IntelliPlant AI

Table of Contents

1. Introduction
 2. Objectives
 3. LangGraph Architecture
 4. State Management
 5. Nodes
 6. Edges
 7. Conditional Routing
 8. Agent Workflows
 9. Parallel Execution
 10. Error Handling
 11. Memory Integration
 12. Human-in-the-Loop
 13. Future Extensions
-

1. Introduction

LangGraph serves as the orchestration engine of IntelliPlant AI. It coordinates multiple specialized agents, manages state, controls workflows, and ensures reliable execution.

Unlike traditional chains, LangGraph enables:

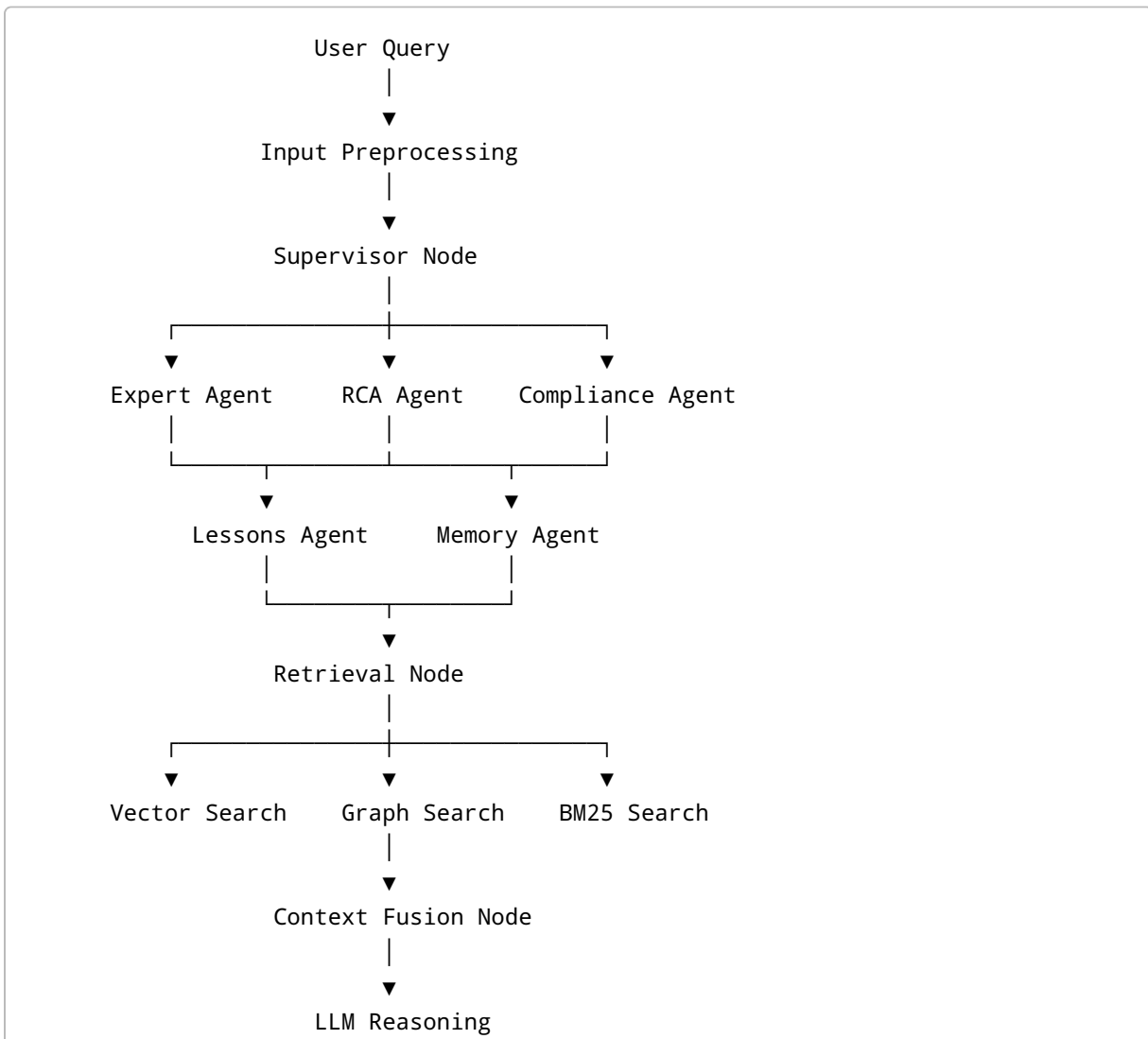
- Stateful execution
- Conditional routing
- Parallel processing
- Retry mechanisms
- Human approval
- Long-running workflows

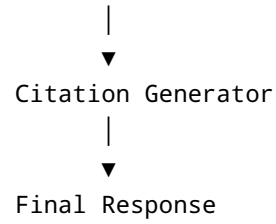
2. Design Objectives

The workflow should:

- Support multiple agents.
 - Maintain context.
 - Allow parallel execution.
 - Handle failures gracefully.
 - Enable explainable reasoning.
 - Support future autonomous workflows.
-

3. Overall Workflow Architecture





4. Graph State

```
class GraphState(TypedDict):  
  
    query: str  
  
    intent: str  
  
    retrieved_docs: list  
  
    graph_context: list  
  
    vector_context: list  
  
    keyword_context: list  
  
    merged_context: str  
  
    memory_context: str  
  
    intermediate_steps: list  
  
    selected_agents: list  
  
    answer: str  
  
    confidence_score: float  
  
    citations: list  
  
    errors: list
```

5. Nodes

Input Node

Responsibilities

- Query normalization
 - Validation
 - Session loading
-

Supervisor Node

Responsibilities

- Intent classification
- Agent selection
- Context initialization

Functions

```
classify_intent()  
  
route_agents()  
  
initialize_state()
```

Expert Agent Node

Handles

- Manuals
- SOPs
- Equipment procedures

Functions

```
retrieve_manuals()  
  
generate_answer()
```

RCA Agent Node

Handles

- Incident reports
- Historical failures

Functions

```
find_similar_failures()  
  
root_cause_analysis()
```

Compliance Agent Node

Handles

- OISD
- Factory Act
- ISO

Functions

```
detect_compliance_gaps()  
  
generate_audit_evidence()
```

Lessons Agent Node

Handles

- CAPA
- Near misses
- Recurring incidents

Functions

```
analyze_patterns()  
  
generate_recommendations()
```

Memory Node

Responsibilities

Retrieve:

- Conversation history
- User preferences
- Previous queries

Sources

Redis

Neo4j

PostgreSQL

Retrieval Node

Coordinates:

Vector Search

Qdrant

Graph Search

Neo4j

BM25 Search

Keyword Search

Context Fusion Node

Merge:

- Vector results
- Graph results
- Keyword results

Rank contexts.

LLM Node

Responsibilities

Reasoning

Summarization

Tool calling

Explanation generation

Citation Node

Responsibilities

Generate:

- Source references
 - Confidence score
-

6. Edges

```
graph TD;
    Input --> Supervisor;
    Supervisor --> Conditional_Routing[Conditional Routing];
    Conditional_Routing --> Agents;
    Agents --> Memory;
    Memory --> Retrieval;
    Retrieval --> Context_Fusion[Context Fusion];
    Context_Fusion --> LLM;
    LLM --> Citation;
```

↓
Response

7. Conditional Routing

Engineering Query

Query
↓
Supervisor
↓
Expert Agent

Root Cause Query

Query
↓
Supervisor
↓
RCA Agent

Compliance Query

Query
↓
Supervisor
↓
Compliance Agent

Mixed Query

Example

"Why Pump P101 failed and does it violate OISD standards?"


```
Supervisor
  ↓
RCA Agent
+
Compliance Agent
  ↓
Merge Outputs
```

8. Parallel Execution

LangGraph supports concurrent nodes.

```
Supervisor
  ↓
Expert Agent
RCA Agent
Compliance Agent
Lessons Agent
  ↓
Aggregation Node
```

Benefits

- Faster responses
- Independent failures
- Better accuracy

9. Retrieval Workflow

```
Retrieval Node
  |
Vector Search
Graph Search
BM25 Search
  |
```

Context Ranking
|
Top-K Results

10. Query Workflow

Example

Query:

"Why did Pump P101 fail?"

Workflow

Input

↓

Supervisor

↓

RCA Agent

↓

Memory Node

↓

Hybrid Search

↓

LLM

↓

Citation

↓

Response

11. Compliance Workflow

Query

"Show OISD violations."

Workflow

Input

↓

Supervisor

↓

Compliance Agent

↓

Retrieve Standards

↓

Gap Analysis

↓

LLM

↓

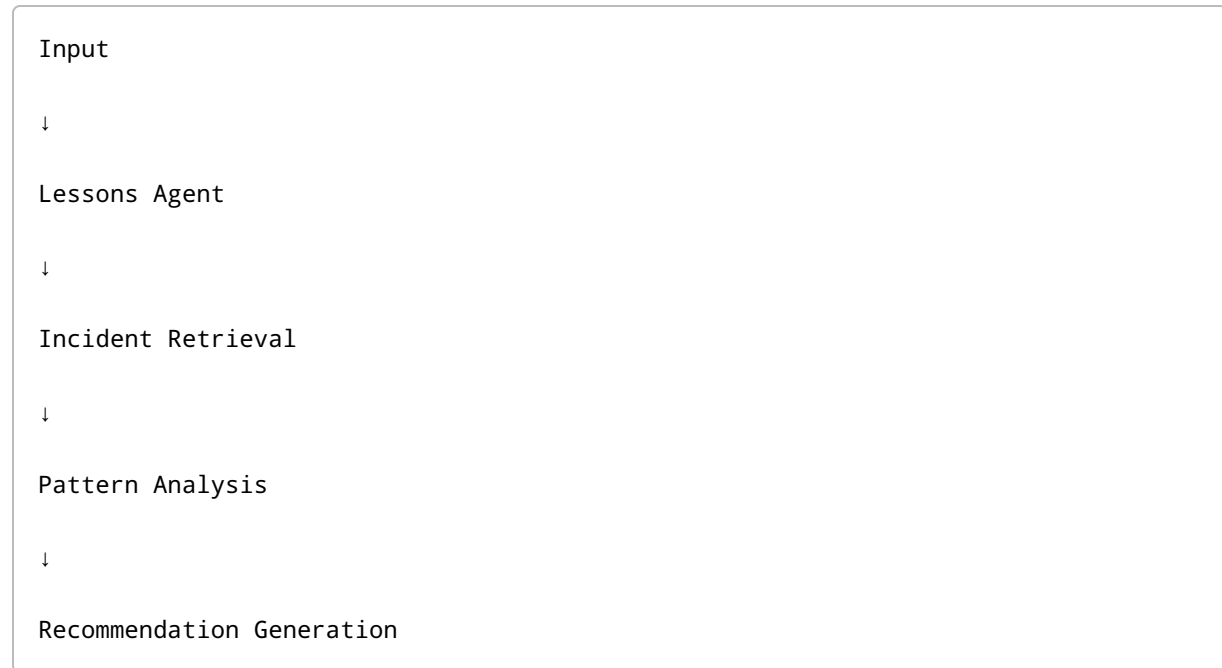
Report

12. Lessons Learned Workflow

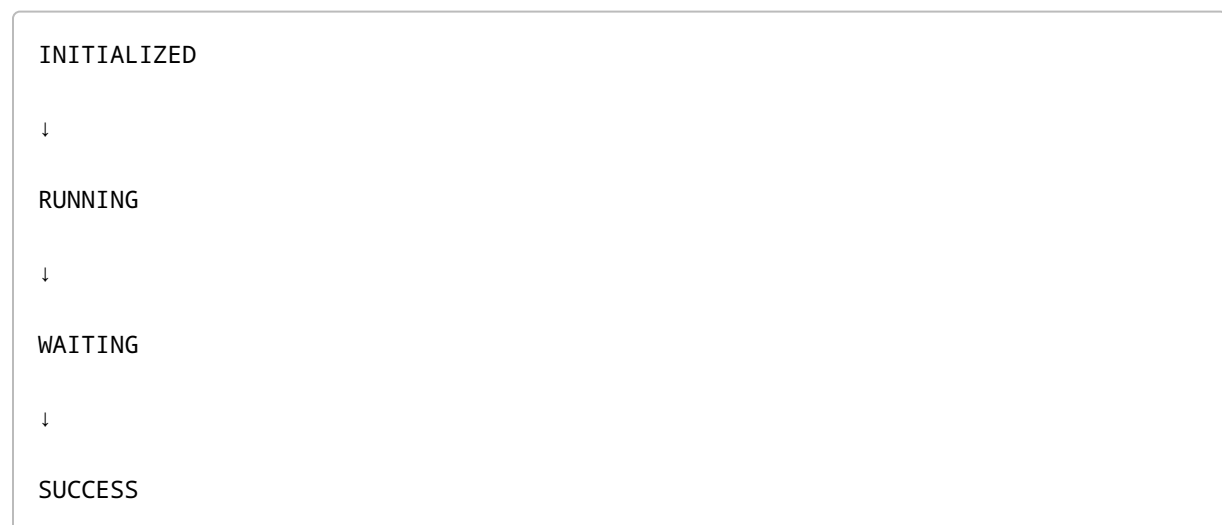
Query

"Show recurring incidents."

Workflow



13. State Transitions



Failure path





14. Memory Integration

Short-Term Memory

Redis

Stores

- Session history
- Recent interactions

Long-Term Memory

Neo4j

Stores

- Historical knowledge

Semantic Memory

Qdrant

Stores

- Embeddings
-

15. Human-in-the-Loop

Critical workflows require approval.

Example

Compliance Report

Compliance Agent

↓

Draft Report

↓

Human Approval

↓

Publish

16. Retry Strategy

Agent Failure

↓

Retry

↓

Fallback Agent

↓

Graceful Degradation

17. Error Handling

LLM Timeout

Fallback Model

Qdrant Failure

BM25 Search

Neo4j Failure

Vector Search Only

Agent Failure

Supervisor Reroutes

18. Observability

Track

- Agent execution time
- Token usage
- Cost
- Errors
- Retry count

Tools

LangSmith

Prometheus

Grafana

19. Example LangGraph Structure

```
workflow = StateGraph(GraphState)

workflow.add_node("supervisor")

workflow.add_node("expert_agent")

workflow.add_node("rca_agent")

workflow.add_node("compliance_agent")

workflow.add_node("lessons_agent")

workflow.add_node("retrieval")

workflow.add_node("context_fusion")

workflow.add_node("llm")

workflow.add_node("citation")

workflow.set_entry_point("supervisor")
```

20. Future Workflows

Predictive Maintenance Agent

Predict equipment failures.

IoT Agent

Process sensor streams.

Digital Twin Agent

Simulate plant operations.

Inventory Agent

Optimize spare parts.

Autonomous Planner Agent

Create maintenance plans.

Workflow Summary

```
graph TD; A[User Query] --> B[Supervisor]; B --> C[Parallel Agents]; C --> D[Memory]; D --> E[Hybrid Retrieval]; E --> F[Context Fusion]; F --> G[LLM Reasoning]; G --> H[Citation Engine]; H --> I[Final Response];
```

User Query
↓
Supervisor
↓
Parallel Agents
↓
Memory
↓
Hybrid Retrieval
↓
Context Fusion
↓
LLM Reasoning
↓
Citation Engine
↓
Final Response

Conclusion

LangGraph acts as the orchestration backbone of IntelliPlant AI, enabling stateful execution, conditional routing, parallel agents, memory integration, and explainable AI. This workflow transforms individual agents into a unified Industrial Operations Brain capable of intelligent reasoning and knowledge discovery.